

Mount Kenya University

Unlocking Infinite Possibilities

NAME: SAMUEL NJUGUNA.

ADM NUMBER: BSCCS/2024/54633.

BACHELOR OF SCIENCE IN COMPUTER SCIENCE, SEP 2024 INTAKE.

UNIT CODE: BIT3208.

LECTURER: MR. MICHAEL NYORO.

YEAR III, SEM II.

PROJECT DETAILS.

NAME: PITSTOP PARTS.

TECHNOLOGIES SELECTED;

i) Frontend.

- ✓ HTML5 for page structure and semantics
- ✓ CSS3 for all custom styling, animations, and transitions
- ✓ Bootstrap 5.3 for the responsive grid system and layout breakpoints
- ✓ Vanilla JavaScript for interactivity

ii) Backend

- ✓ PHP 8 for all server-side logic, form handling, session management, and routing.

iii) Database

- ✓ MySQL running through XAMPP on localhost.

TABLE OF CONTENTS.

Week 1: Local Environment Setup

Installation and Testing of Local Development Environment	4
Reflections for Week 1	6

Week 2: Wireframes and GUI Design

User Interface Planning and System Design.....	7
Reflections for Week 2	10

Week 3: JavaScript and PHP Basics

Frontend Interaction and Backend Foundations	10
Reflections for Week 3	12

Week 4: Server-Side Components and Backend Development

Reflections for Week 4	14
------------------------------	----

Week 5: Database Components and CRUD Operations

Reflections for Week 5	16
------------------------------	----

WEEK ONE: LOCAL ENVIRONMENT SETUP.

Installation and Testing of Local Development Environment.

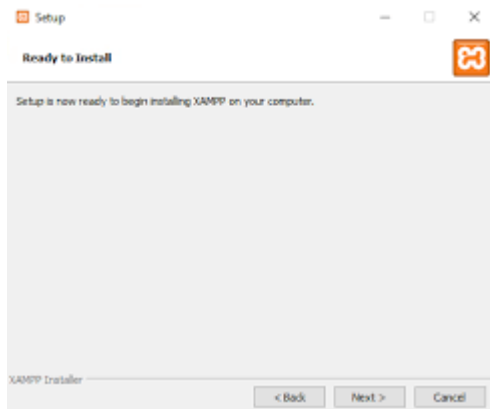


Fig 1: XAMPP Installation.

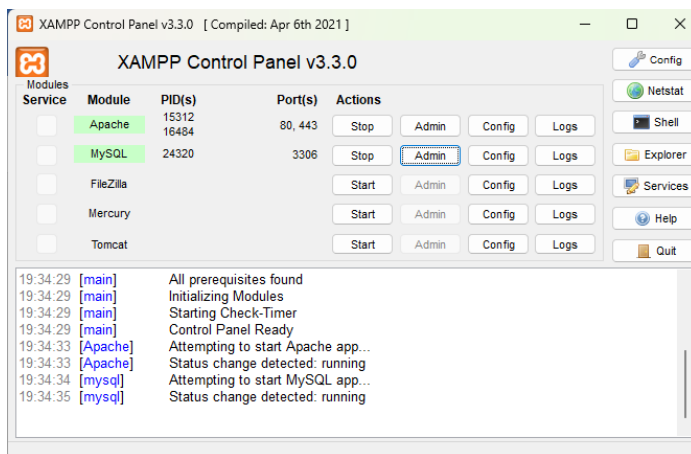


Fig 2: Apache and MySQL Running.

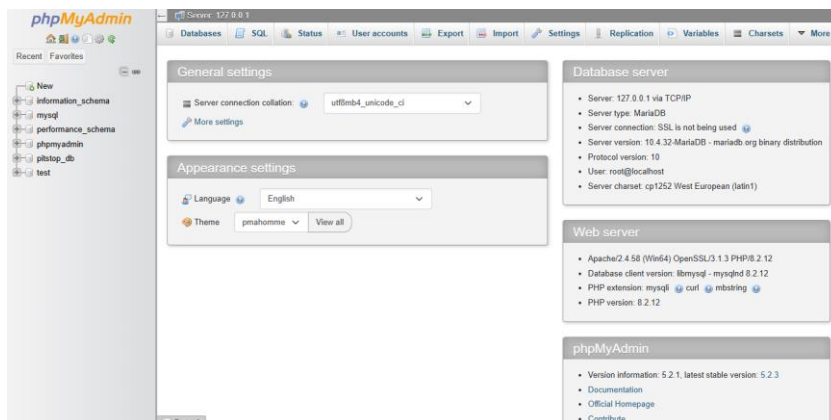


Fig 3: XAMPP Dashboard Page.

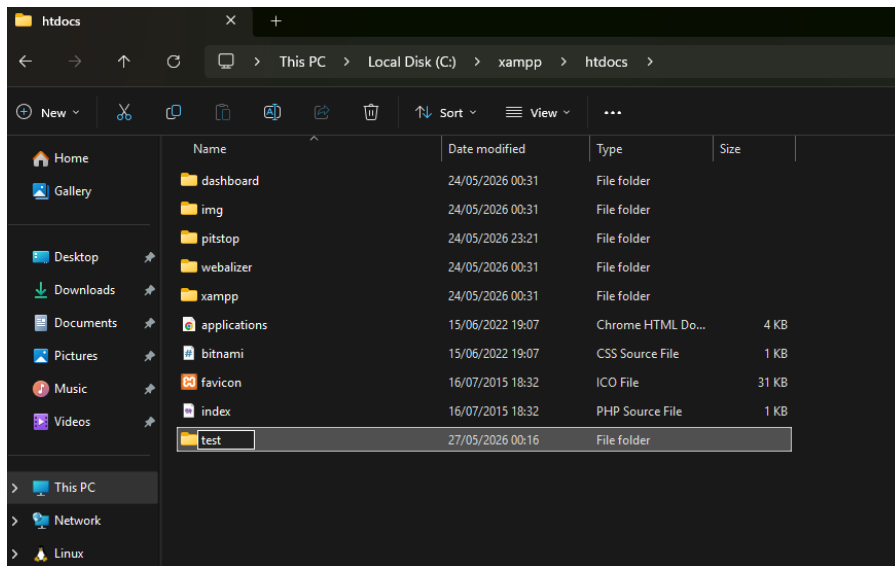


Fig 4: Test folder inside XAMPP htdocs folder.

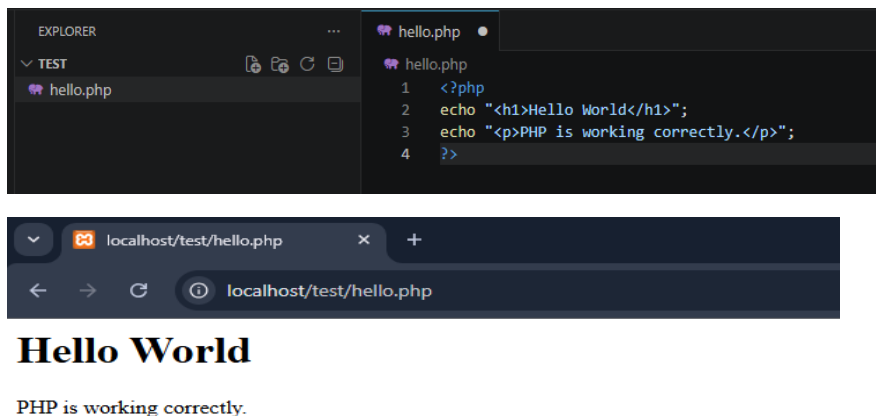
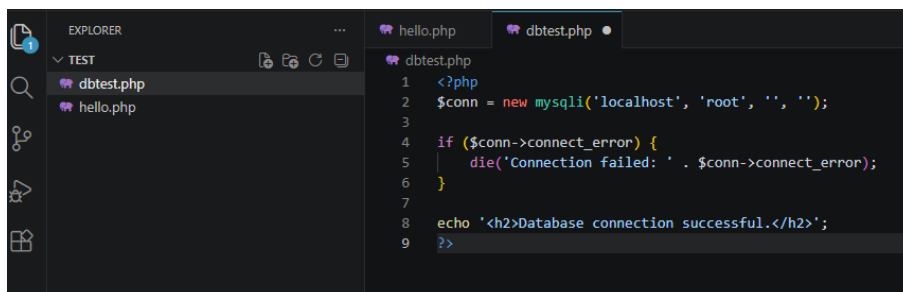


Fig 5: First PHP Page.



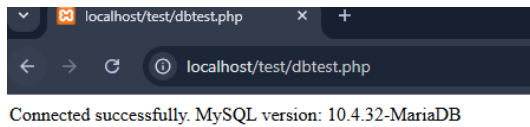


Fig 6: Successful Database Connection.

MY REFLECTIONS FOR WEEK ONE.

Lessons learned;

I learned how to configure a local development environment using XAMPP and how Apache serves PHP files strictly from the htdocs directory.

Technologies used;

I used XAMPP, Apache, MySQL, PHP 8.2.12, and VS Code as my code editor.

Challenges encountered;

My project folder was initially placed inside C:\xampp instead of C:\xampp\htdocs, which caused PHP files to render as blank white pages. I also discovered that plain HTML renders correctly while PHP requires the Apache module to be properly serving from htdocs.

Screenshots;

Fig 1 – Successful XAMPP Installation.

Fig 2 – Apache and MySQL running.

Fig 3 – XAMPP Dashboard running on <http://localhost/>

Fig 4&5 – Test page file inside XAMPP htdocs folder and first successful PHP page.

Fig 6 – Successful database connection.

WEEK 2: WIREFRAMES AND GUI DESIGN.

User Interface Planning and System Design.

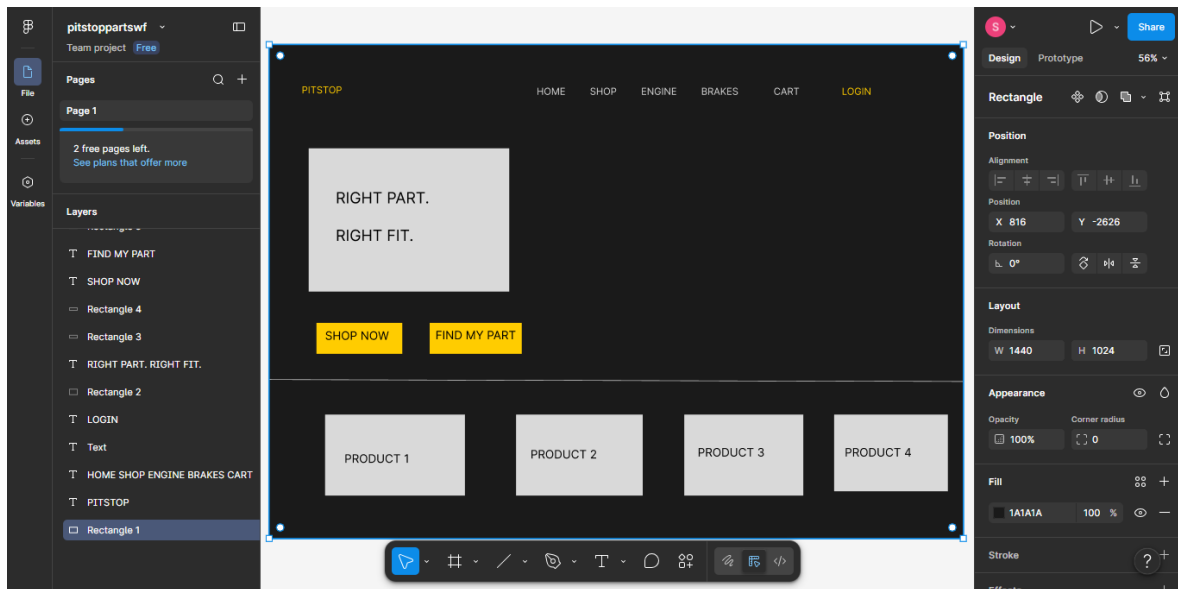


Fig 1: Landing page prototype designed in Figma.com.

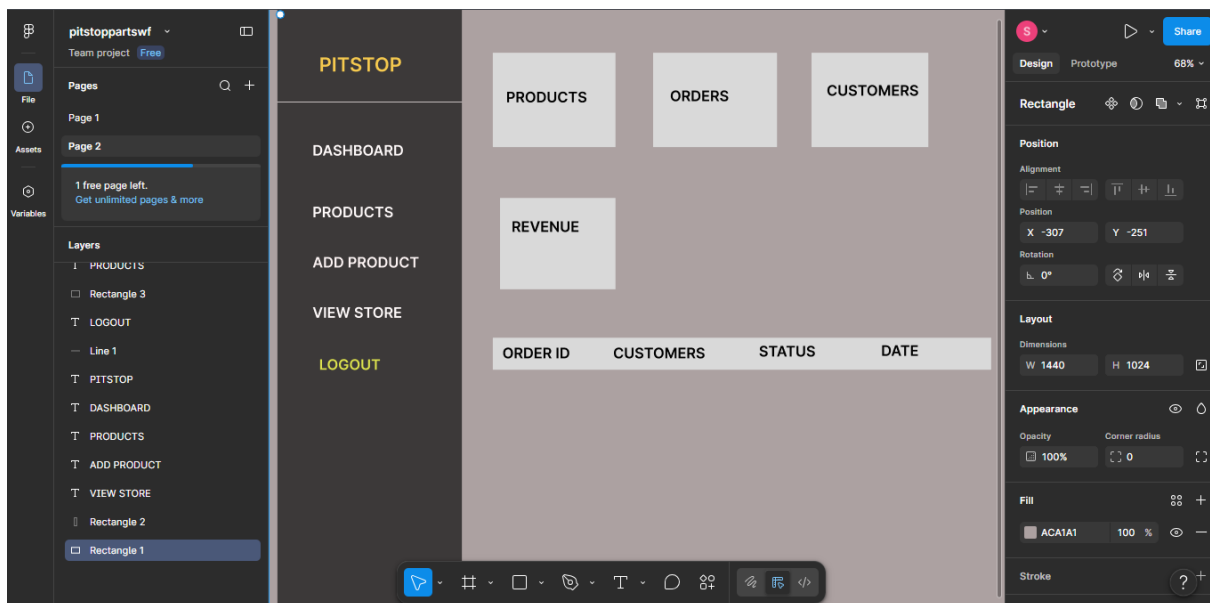


Fig 2: Admin page wireframe.

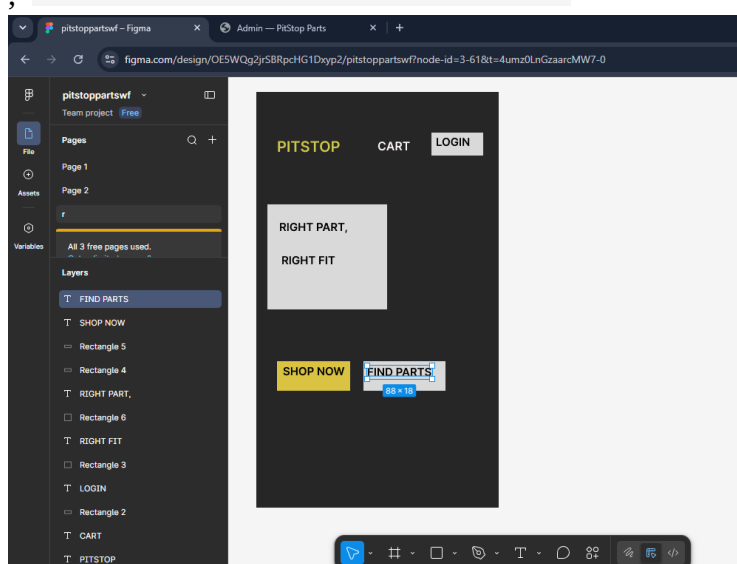
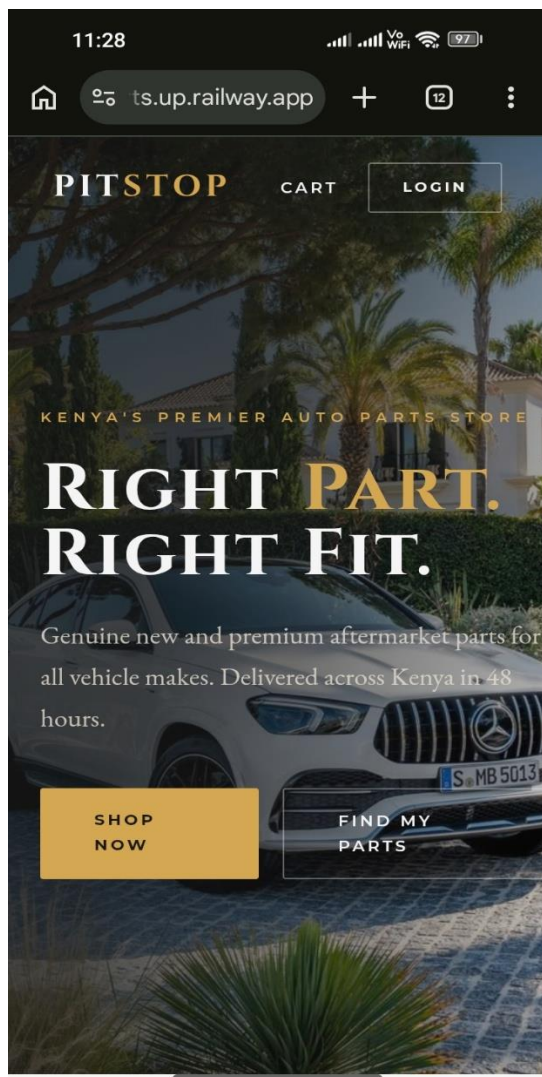


Fig 3: Mobile view mock up, showing mobile compatibility.

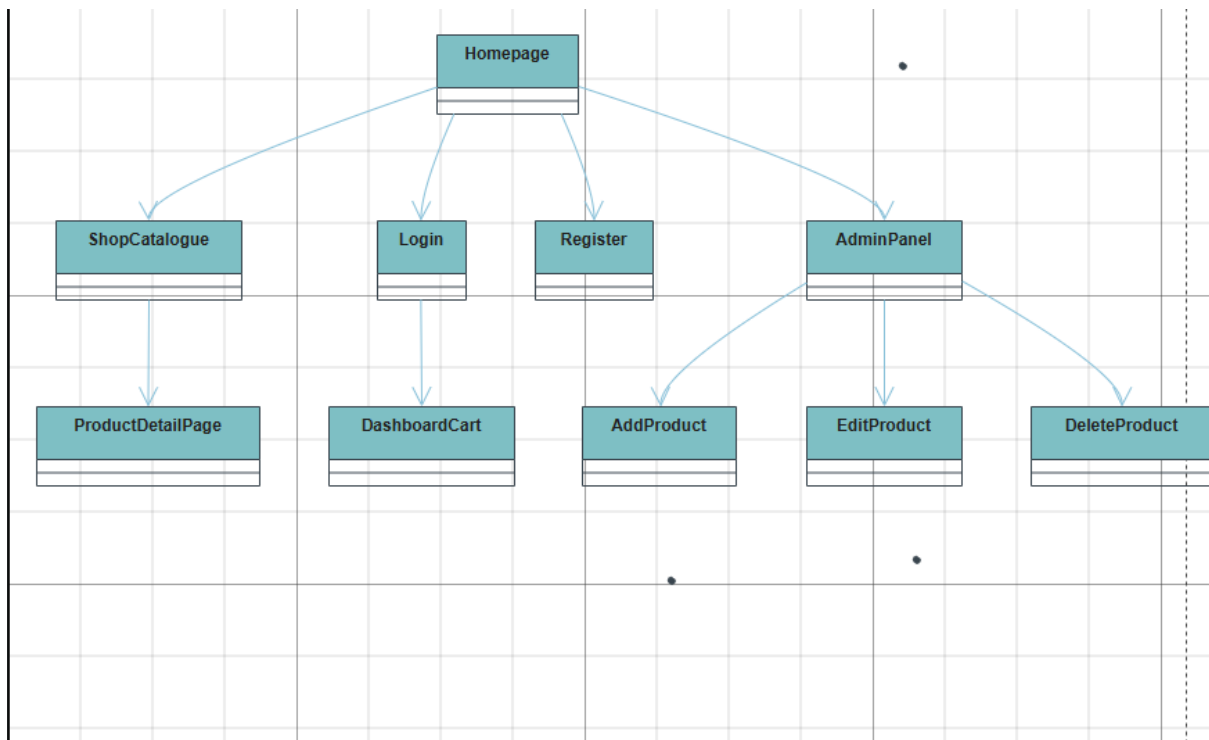
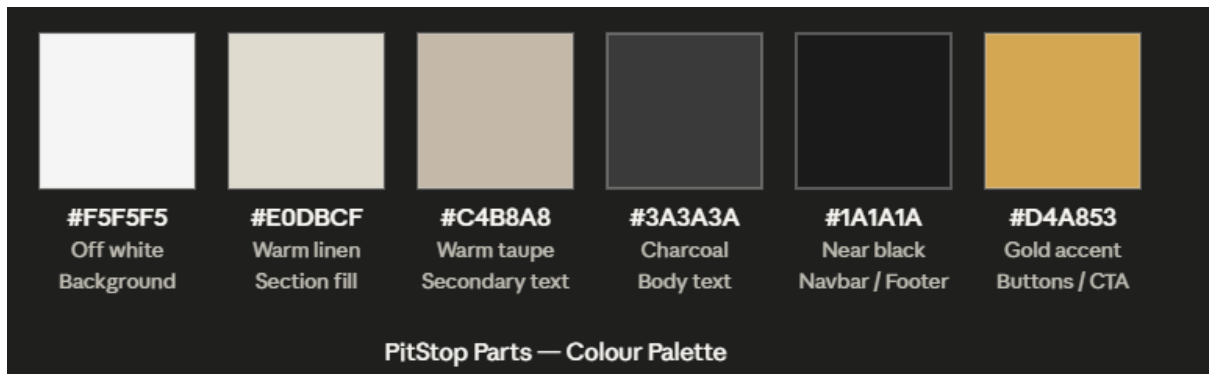


Fig 4: Color Scheme and simple navigation structure.

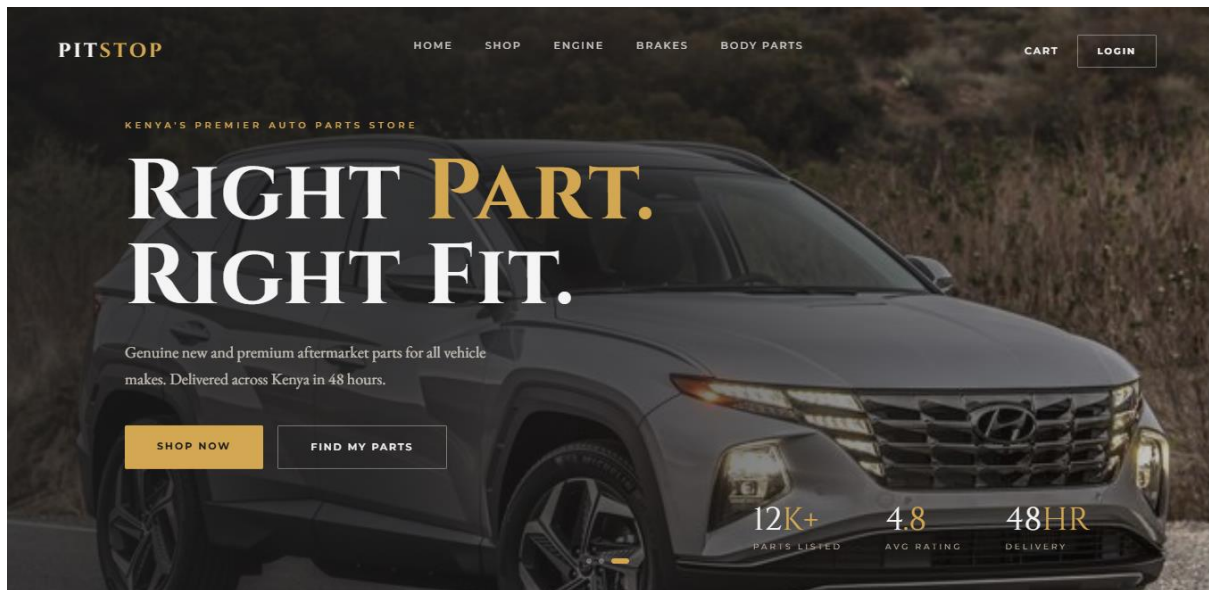
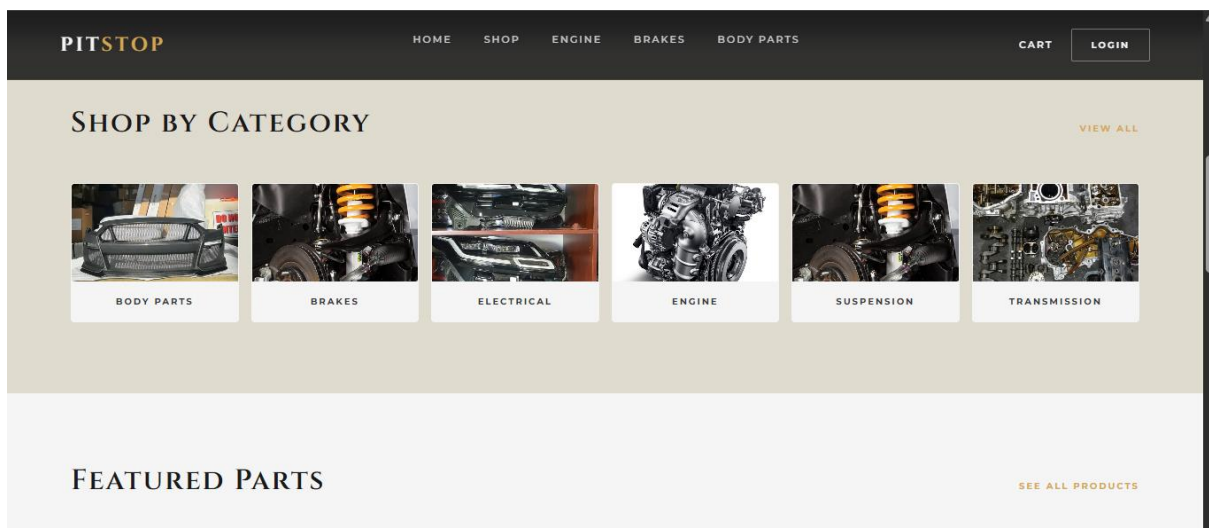


Fig 5: Actual landing page.



MY REFLECTION ON WEEK 2;

The design was selected to be eye-catching for the user, especially the gold accents. This will be enhanced by the changing hero background images to be incorporated.

The goal was to enable the customer to be able to arrive at checkout after 4 actions, making the process quicker.

Intended system users are logged in customers, guests (They can still view store then login in to shop) and admin who is in charge of all CRUD operations.

Week 2 screenshots;

Fig 1 – Figma designed wireframe showing the navbar, Shop Now and Find Part buttons.

Fig 2 – Admin page wireframe layout.

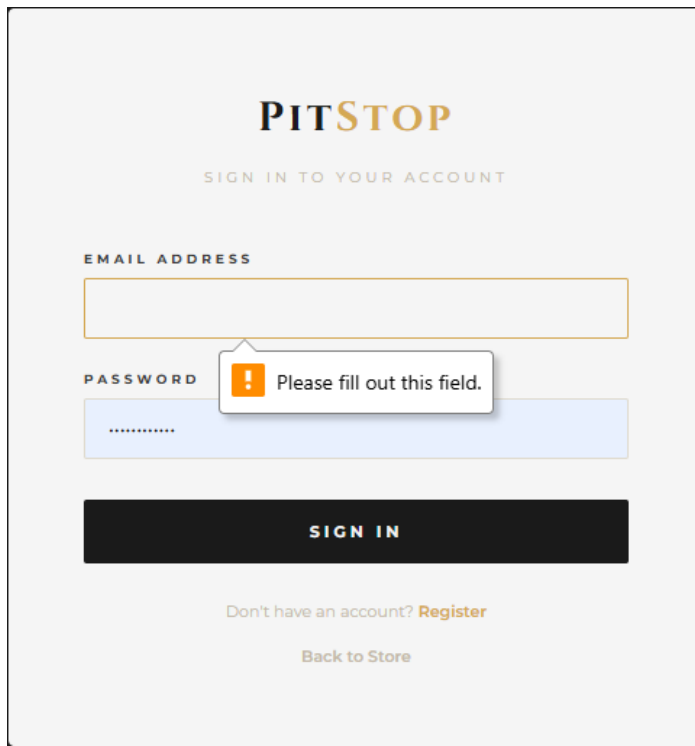
Fig 3 – Mobile view mockup demonstrating responsive compatibility.

Fig 4 – Color scheme and navigation structure planning.

Fig 5 –Actual landing page.

WEEK 3: JAVASCRIPT AND PHP BASICS.

Frontend Interaction and Backend Foundations.



The image shows a login form for a website called 'PITSTOP'. The form is titled 'SIGN IN TO YOUR ACCOUNT'. It has two input fields: 'EMAIL ADDRESS' and 'PASSWORD'. The 'EMAIL ADDRESS' field is empty. The 'PASSWORD' field is filled with dots. A validation error message is displayed above the password field: 'Please fill out this field.' Below the password field is a 'SIGN IN' button. At the bottom of the form, there is a link 'Don't have an account? Register' and a link 'Back to Store'.

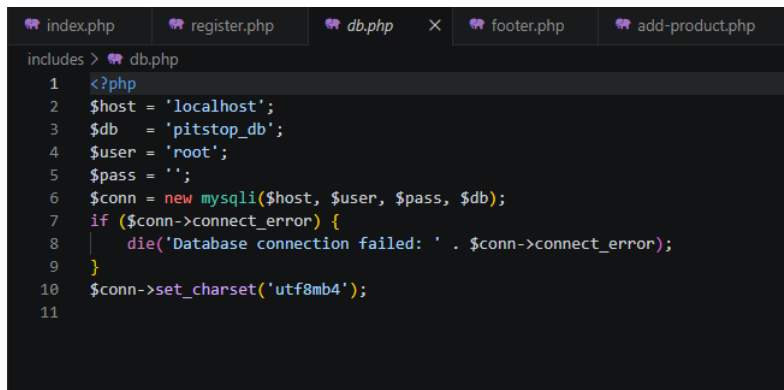
Fig 1: Failed login due to missing credentials, demonstrating proper form validation by JavaScript.

The image shows a web form for creating an account at PITSTOP. At the top, the logo 'PITSTOP' is displayed in a bold, sans-serif font, with 'PIT' in black and 'STOP' in orange. Below the logo, the text 'CREATE YOUR ACCOUNT' is centered in a smaller, black, all-caps font. The form consists of several input fields, each with a label above it: 'FULL NAME' (containing 'Sammy Njuguna'), 'EMAIL ADDRESS' (containing 'njugunas357@gmail.com'), 'PHONE NUMBER' (containing '0727501503'), 'PASSWORD', and 'CONFIRM PASSWORD'. A red error message box is positioned at the top left, stating 'Password must be at least 6 characters.' At the bottom of the form is a black button with the white text 'CREATE ACCOUNT'.

Fig 2: Password strength checker, indicating that password must be 6 characters or longer.

```
function get_categories($conn) {  
    $result = $conn->query("SELECT * FROM categories ORDER BY name ASC");  
    return $result->fetch_all(MYSQLI_ASSOC);  
}  
  
function get_makes($conn) {  
    $result = $conn->query("SELECT * FROM makes ORDER BY name ASC");  
    return $result->fetch_all(MYSQLI_ASSOC);  
}  
  
function get_models_by_make($conn, $make_id) {  
    $id = (int)$make_id;  
    $result = $conn->query("SELECT * FROM models WHERE make_id = $id ORDER BY name ASC");  
    return $result->fetch_all(MYSQLI_ASSOC);  
}
```

Fig 3: Snippet from functions.php. These demonstrate PHP syntax including functions, parameters, return values, and SQL queries.



```
index.php  register.php  db.php  footer.php  add-product.php
includes > db.php
1  <?php
2  $host = 'localhost';
3  $db   = 'pitstop_db';
4  $user = 'root';
5  $pass = '';
6  $conn = new mysqli($host, $user, $pass, $db);
7  if ($conn->connect_error) {
8      die('Database connection failed: ' . $conn->connect_error);
9  }
10 $conn->set_charset('utf8mb4');
11
```

Fig 4: Screenshot showing the mysqli connection using localhost, root, and the database name.

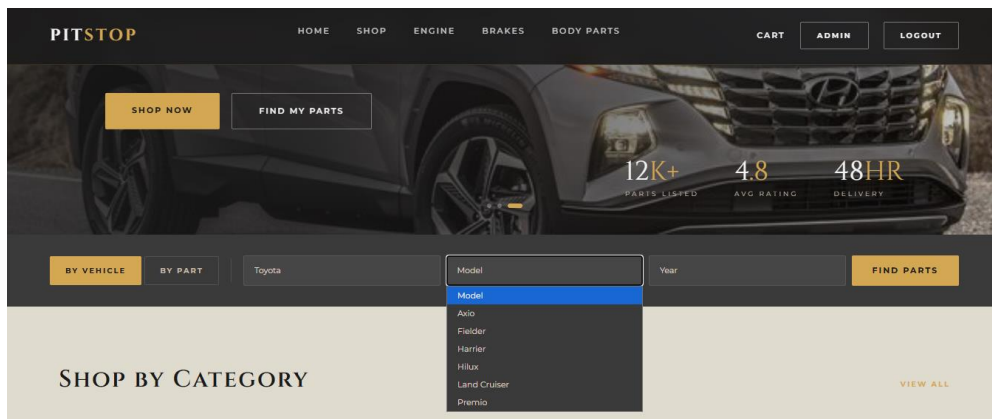


Fig 5: Dynamic model dropdown on the homepage populated via a JavaScript fetch request to get-models.php, which queries the MySQL models table and returns JSON data based on the selected vehicle make. Toyota returns corresponding Toyota models.

MY WEEK THREE REFLECTIONS.

Technologies used: JavaScript for form validation and dynamic dropdowns, PHP for backend functions and database queries, MySQL for storing and retrieving vehicle and product data, and the Fetch API for AJAX communication between the frontend and backend.

JavaScript running in the browser sends requests to PHP files on the server. PHP processes those requests, queries MySQL, and returns data which JavaScript then uses to update the page.

Why database connection matters Without the mysqli connection in db.php no page in PitStop can display products, validate login credentials, or process orders.

Week 3 screenshots;

Fig 1 – Failed login attempt demonstrating JavaScript form validation.

Fig 2 – Password strength checker enforcing a minimum of 10 characters.

Fig 3 – Snippet from functions.php showing PHP functions, parameters, return values, and SQL queries.

Fig 4 – mysqli database connection using localhost, root, and the database name.

Fig 5 – Dynamic model dropdown on the homepage populated via a JavaScript Fetch request to get-models.php, returning JSON data based on selected vehicle make.

WEEK 4: SERVER-SIDE COMPONENTS AND BACKEND DEVELOPMENT.

Fig 1: Screenshot of the PHP section that handles the POST request. This demonstrates PHP receiving and processing form data from the browser.

The image displays two side-by-side screenshots of the PITSTOP web application's authentication forms. Both forms have a light gray background and the PITSTOP logo at the top.

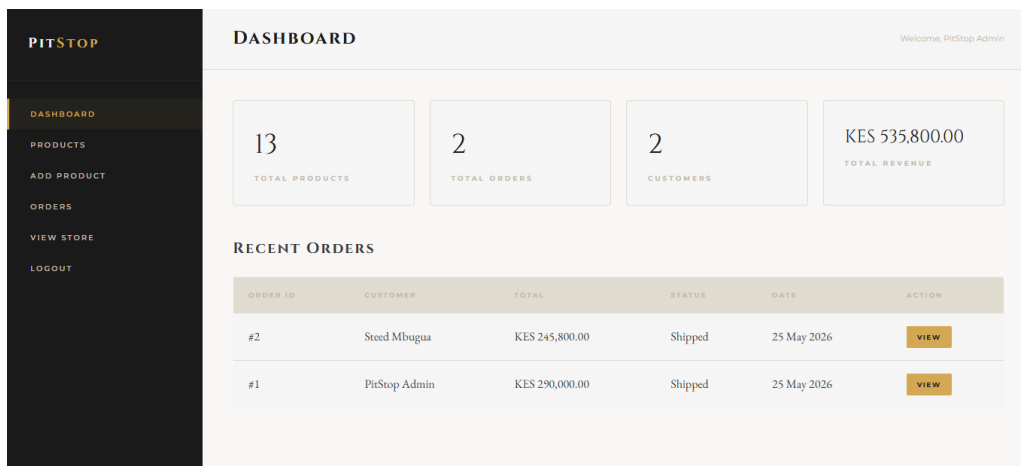
Left Screenshot (Sign In):

- Header: PITSTOP, SIGN IN TO YOUR ACCOUNT
- Form Fields: EMAIL ADDRESS (empty), PASSWORD (empty with a vertical cursor).
- Buttons: A black button labeled SIGN IN.
- Footer: Don't have an account? Register (in orange), Back to Store (in gray).

Right Screenshot (Create Account):

- Header: PITSTOP, CREATE YOUR ACCOUNT
- Form Fields: FULL NAME (empty), EMAIL ADDRESS (pre-filled with admin@pitstopparts.co.ke), PHONE NUMBER (empty), PASSWORD (empty with masked characters), CONFIRM PASSWORD (empty).
- Buttons: A black button labeled CREATE ACCOUNT.
- Footer: Already have an account? Login (in orange).

Fig 2: HTML forms using the POST method to send data to the server.



```

39     <?php if (is_logged_in()): ?>
40         <?php if (is_admin()): ?>
41             <a href="<?php echo $base; ?>/admin/index.php" class="nav-btn">Admin</a>
42         <?php endif; ?>
43         <a href="<?php echo $base; ?>/logout.php" class="nav-btn">Logout</a>
44     <?php else: ?>
45         <a href="<?php echo $base; ?>/login.php" class="nav-btn">Login</a>
46     <?php endif; ?>
47 </div>
48 </div>
49 </nav>

```

Fig 3: Session-Based Authentication, admin logged in. This demonstrates session-based authentication. PHP stored user ID and role in \$_SESSION after verifying the password.

CREATE YOUR ACCOUNT

FULL NAME

Steed Mbugua

EMAIL ADDRESS

steedm@gmail.com

PHONE NUMBER

0734234565

PASSWORD

Medium password

CONFIRM PASSWORD

CREATE ACCOUNT

Already have an account? [Login](#)

	id	name	email	password	phone	role	created_at
☐ Edit Copy Delete	1	PitStop Admin	admin@pitstopparts.co.ke	\$2y\$10\$92IXUNpkj0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2...	NULL	admin	2026-05-24 23:24:10
☐ Edit Copy Delete	2	Steed Mbugua	steedm@gmail.com	\$2y\$10\$Pv8fTA7s22F5oalFx3nee.hFLqoP66szGLgJie19eUu...	0734234565	customer	2026-05-25 13:24:09
☐ Edit Copy Delete	3	Sammy Njuguna	njugunas357@gmail.com	\$2y\$10\$S6iAOhhAvj1j1.mxEQMA5ukjLO5YDBHMwdPgEDOUb2....	0727501503	customer	2026-05-28 23:08:26

Fig 4: User 'Steed' added to database after registration. This proves the full cycle form submitted, PHP processed it, MySQL stored it.

MY WEEK 4 REFLECTIONS;

Technologies used I used PHP for form processing and session management, MySQL for storing registered user records, and the password_hash function for secure credential storage. The mysqli prepared statements were used to prevent SQL injection on all form inputs.

The \$_SESSION keeps the user authenticated across the homepage, shop, cart, and admin panel for the duration of their browser session, preventing frequent logins over and over again.

Week 4 screenshots;

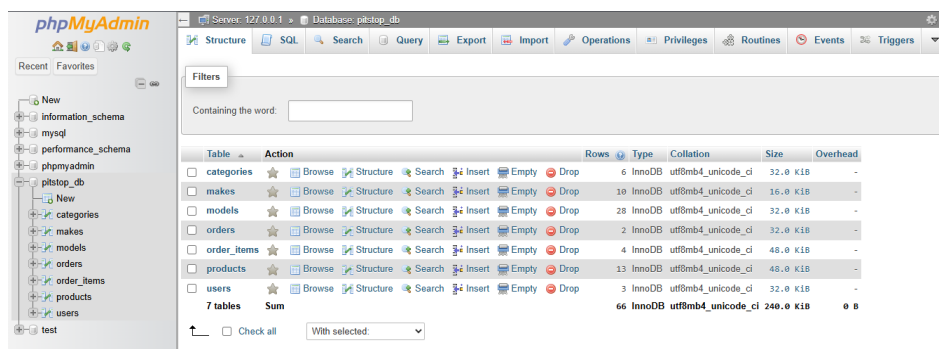
Fig 1 – PHP section handling the POST request, demonstrating server-side form data processing.

Fig 2 – HTML form using the POST method to send data to the server.

Fig 3 – Session-based authentication with admin logged in, showing \$_SESSION storing user ID and role after password verification.

Fig 4 – User 'Steed' added to the database after registration, confirming the full form-to-database cycle.

WEEK 5: DATABASE COMPONENTS AND CRUD OPERATIONS.



The screenshot shows the phpMyAdmin interface for a database named 'pitstop_db'. The left sidebar shows the database structure, and the main area displays a table of 7 tables. The table has columns for Table, Action, Rows, Type, Collation, Size, and Overhead.

Table	Action	Rows	Type	Collation	Size	Overhead
categories	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
makes	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
models	Browse Structure Search Insert Empty Drop	28	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
orders	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
order_items	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
products	Browse Structure Search Insert Empty Drop	13	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
users	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
7 tables	Sum	66	InnoDB	utf8mb4_unicode_ci	240.0 KiB	0 B

Fig 1: Database created with 7 tables.

```
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  phone VARCHAR(20),
  role ENUM('customer','admin') DEFAULT 'customer',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  slug VARCHAR(100) NOT NULL UNIQUE,
  image_url VARCHAR(255)
);
```

Fig 2: Sample SQL queries to create tables.

PITSTOP

- DASHBOARD
- PRODUCTS
- ADD PRODUCT**
- ORDERS
- VIEW STORE
- LOGOUT

Brakes

PRICE (KES) *

25400

STOCK QUANTITY

12

CONDITION

Refurbished

DESCRIPTION

PRODUCT IMAGE (JPG, PNG, WEBP — MAX 5MB)

Choose File images (1).jpg

☒ Mark as Featured Product

ADD PRODUCT

Fig 3: Adding product to shop(create).

PRODUCTS **ADD NEW PRODUCT**

IMAGE	NAME	PART NO	CATEGORY	PRICE	STOCK	CONDITION	FEATURED	ACTIONS
	Brake Pads	Brakepads23455	Brakes	KES 25,400.00	12	Refurbished	No	EDIT DELETE

Fig 4: Item 'brake pads' added to shop(create+read).

PRODUCTS **ADD NEW PRODUCT**

localhost says
Delete this product?

OK **Cancel**

IMAGE	NAME	PART NO	CATEGORY	PRICE	STOCK	CONDITION	FEATURED	ACTIONS
	Brake Pads	Brakepads23455	Brakes	KES 25,400.00	12	Refurbished	No	EDIT DELETE

Fig 5: Item 'brake pads' deleted.

WEEK 5 LESSONS.

Technologies used I used MySQL for the relational database, phpMyAdmin for visual database management, PHP with mysqli prepared statements for all database queries, and the admin panel as the frontend interface for CRUD operations.

Every e-commerce system is built on CRUD. It allows the admin to list new parts, Read powers the product catalogue customers browse, Update keeps prices and stock levels accurate, and Delete removes discontinued parts from the system.

Week 5 screenshots;

Fig 1 – Database created with 7 tables.

Fig 2 – Sample SQL queries used to create the tables.

Fig 3 – Adding a product to the shop, demonstrating the Create operation.

Fig 4 – Item 'brake pads' successfully added, demonstrating Create and Read.

Fig 5 – Item 'brake pads' deleted, demonstrating the Delete operation.

GITHUB LINK; <https://github.com/sammy1-com/pitstop>

LIVE HOSTING LINK; <https://pitstopparts.up.railway.app>